PRACTICAL 07

Aim : Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Code :

```
~
  GNU nano 8.7
#include <stdio.h>

#define P 5    // Processes
#define R 4    // Resources

int main() {

    int alloc[P][R], max[P][R], need[P][R];
    int avail[R];
    int finish[P] = {0};
    int safeSeq[P];

    char *process[P] = {
        "GoogleDrive",
        "Firefox",
        "WordProcessor",
        "Excel",
        "PowerPoint"
    };

    int i, j, k;

    // -------- Allocation Matrix --------
    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<P;i++)
        for(j=0;j<R;j++)
            scanf("%d",&alloc[i][j]);

    // -------- Maximum Matrix --------
    printf("\nEnter Maximum Matrix:\n");
    for(i=0;i<P;i++)
        for(j=0;j<R;j++)
            scanf("%d",&max[i][j]);

    // -------- Available Resources --------
    printf("\nEnter Available Resources:\n");
    for(j=0;j<R;j++)
        scanf("%d",&avail[j]);

    // -------- Need Matrix --------
    for(i=0;i<P;i++)
        for(j=0;j<R;j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("\nNeed Matrix:\n");
    for(i=0;i<P;i++){
        printf("%s: ",process[i]);
        for(j=0;j<R;j++)
            printf("%d ",need[i][j]);
        printf("\n");
    }

    // -------- Banker's Algorithm --------
    int count = 0;
```

```
// -------- Banker's Algorithm --------
int count = 0;

while(count < P) {
    int found = 0;

    for(i=0;i<P;i++) {

        if(finish[i] == 0) {

            for(j=0;j<R;j++)
                if(need[i][j] > avail[j])
                    break;

            if(j == R) {

                for(k=0;k<R;k++)
                    avail[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("\nSystem is NOT in safe state!\n");
        return 0;
    }
}

// -------- Safe Sequence --------
printf("\nSystem is in SAFE state.\nSafe sequence:\n");

for(i=0;i<P;i++)
    printf("%s -> ",process[safeSeq[i]]);

printf("END\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Output:

```
vidhi@VidhiKore MSYS ~
$ nano banker.c

vidhi@VidhiKore MSYS ~
$ gcc banker.c -o banker

vidhi@VidhiKore MSYS ~
$ ./banker

Enter Allocation Matrix:




0 1 0 3
2 0 0 1
3 0 2 1
2 1 1 0
0 0 2 2

Enter Maximum Matrix:
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3

Enter Available Resources:
3 3 2 2

Need Matrix:
GoogleDrive: 7 4 3 1
Firefox: 1 2 2 1
WordProcessor: 6 0 0 1
Excel: 0 1 1 2
PowerPoint: 4 3 1 1

System is in SAFE state.
Safe sequence:
Firefox -> Excel -> PowerPoint -> GoogleDrive -> WordProcessor -> END

vidhi@VidhiKore MSYS ~
$
```